Qwen

From Grammar to Governance: A Blueprint for Safe, Automated Workload Discovery in the EcoNet Constellation

Defining Safe Computation: The `NonActuatingWorkload` Trait and Its Mathematical Guarantees

The formalization of the `NonActuatingWorkload` trait represents the foundational step in establishing a high-integrity computational layer within the EcoNet constellation. It moves the concept of a "safe" workload from an operational guideline to a machine-verifiable contract enforced through Rust's type system and a suite of mathematical proofs. The strategic priority of defining this trait before building discovery and automation tools ensures that all subsequent layers operate on a stable, predictable, and rigorously defined grammar . The objective is to create a framework that guarantees, for any valid input, that the resulting output remains within the bounds of the established ecosafety model, thereby preventing unintended consequences from propagating through the system. This approach directly addresses the need for a frozen, high-integrity foundation that maintains $K \approx 0.95$, $E \approx 0.92$, and $R \approx 0.12$ across the constellation .

The core of this formalization is the definition of the `NonActuatingWorkload` trait family within the EcoNet SPINE repository . This trait serves as a blueprint for pure kernels that transform input data into updated risk metrics without engaging any actuator logic, aligning perfectly with the project's scope of focusing on verification and metadata rather than runtime control . The proposed structure for this trait is designed for maximum expressiveness and compile-time safety by tying its associated types directly to the ALN schemas that define the data it consumes and produces. This creates a typed interface where the dependencies of a workload are explicitly declared and verifiable by the compiler, reducing the risk of incorrect usage or data corruption.

```
// In ecosafety core (SPINE)
pub trait NonActuatingWorkload {
    /// Typically an ALN-backed shard or telemetry snapshot.
    type Input;
```

```
    /// Must include full RiskVector, residual, and KER window.
    type Output;

    /// Pure, non-actuating kernel: telemetry -> (r, V_t, K, E, R, lane, k
    fn execute(&self, input: Self::Input) -> Self::Output;
}
```

This trait signature establishes a clear contract. The `type Input` represents the prerequisite data, such as a telemetry snapshot or a specific ALN-backed shard like `HydrologicalBufferPhoenix2026v1` . The `type Output` is constrained to provide a comprehensive result, mandatorily including the full updated `RiskVector`, the Lyapunov residual `V_t`, and the KER window metrics (`kmetric`, `emetric`, `rmetric`) . The `execute` method itself is defined as a pure function, consuming an input and producing an output, with no side effects or access to external state beyond what is provided in the input . This structure allows for a fine-grained catalog of functions, enabling agents to discover kernels based on their specific input/output requirements, such as finding all `NonActuatingWorkload`s that accept a `FlowVacSubstrateShard.v1` and produce a `RiskVector`.

The true power of the `NonActuatingWorkload` trait lies not just in its structure but in the set of mathematical invariants it enforces. These invariants are not merely suggestions; they are executable proofs that must be validated for every implementation. They form the bedrock of the safety guarantees, ensuring that the outputs of these workloads are always consistent with the underlying ecosafety model. Three primary invariants must be proven: Boundedness, Monotonicity in Harmful Directions, and the Safestep Invariant under corridor constraints.

First, the **Boundedness** invariant is fundamental to the mathematical validity of the entire system. It requires that for any valid input, every component of the resulting risk vector, `r'_j`, is strictly contained within the `[0,1]` interval . This directly implies that the updated Lyapunov residual, `V_t' = Σ w_j * (r'_j)²`, is always a non-negative value . Proving boundedness involves writing unit tests that systematically explore the input space, particularly at its boundaries, to ensure that no workload can generate a risk coordinate outside the predefined safe limits. For example, a test for a hydraulic decay kernel would verify that increasing the contaminant load parameter does not push `r_hydraulics` above 1.0 or below 0.0. This property is essential because it preserves the convexity and non-negativity of the Lyapunov function, which are prerequisites for proving system stability.

Second, the **Monotonicity in Harmful Directions** invariant ensures logical consistency between inputs and outputs. If a scalar input parameter representing a "harmful" condition increases (e.g., time since last maintenance, concentration of a toxin), the corresponding risk coordinate `r_j` must be non-decreasing . Conversely, if the harmful parameter decreases, the risk coordinate must be non-increasing. This prevents counterintuitive behavior where a worse physical state leads to a lower calculated risk. Proving monotonicity requires a more sophisticated testing strategy than simple boundary checks. Unit tests must perform sweeps across the input parameters, holding all but one constant, and verify that the relationship between the harmful parameter and the corresponding risk coordinate is monotonically non-decreasing. For instance, a `MaterialKinetics` workload would be tested to confirm that as `rmassloss` increases, so too must the overall material degradation risk. This invariant ensures that the workloads serve their purpose as accurate measures of risk rather than arbitrary mathematical functions.

Third, the **Safestep Invariant** combines the static properties of the risk coordinates with the dynamic constraints of the system's corridors. It states that when a `NonActuatingWorkload` is applied to an admissible state in a permitted lane (such as RESEARCH or PROD), and the resulting state is clipped to stay within the defined safety corridors, the Lyapunov residual cannot increase: `V_{t+1} ≤ V_t` . This is the core dynamic safety guarantee. It ensures that even if a workload calculation temporarily pushes a risk coordinate outside its gold band, the enforcement of the corridor brings the system back to a safer or equivalent state, preventing the accumulation of risk over time. Verifying this invariant requires simulating workload executions and corridor clipping. Tests would start from various points within the admissible region, apply a workload, and then check if the final `V_t` after corridor enforcement is less than or equal to the initial `V_t`. This proof is critical for demonstrating that the system has a mechanism to self-correct and maintain stability.

Once these traits are formally defined and their invariants are codified in tests, the next phase is to retrofit existing non-actuating crates to conform to this new standard. Crates such as `EnergyToMassKernels`, `MaterialKinetics`, `rsigma` data-quality kernels, `HydraulicDecayFrame`, `QualityMixingFrame`, and `FoulingForecastFrame` are prime candidates for implementation . This process involves several steps for each crate. First, the public API of the crate must be examined to identify the core computational function that maps inputs to outputs. Second, this function must be adapted to fit the `NonActuatingWorkload` trait's signature, likely by creating a wrapper struct that holds the necessary configuration and implements the `execute` method. Third, the comprehensive suite of unit tests for boundedness, monotonicity, and safestep must be written or adapted for the new implementation. This effort transforms a collection of

standalone functions into a cohesive, verified library of safe computational kernels. The outcome is a significant increase in the Knowledge-factor (K), as a prose concept is replaced with a machine-verifiable contract, and an increase in Eco-impact (E), as AI agents can now reliably discover and use these verified kernels with confidence .

Finally, the ultimate utility of this formalization is realized when it is leveraged by higher-level discovery tools. The `econetrepomanifest` crate and its FFI interfaces will use the existence of the `NonActuatingWorkload` trait to automatically populate an agent function catalog . By scanning the dependency graph for crates that implement this trait, the manifest tool can dynamically generate a searchable index of all safe, discoverable functions in the constellation. This catalog would list functions by name, the crate they reside in, their specific input and output ALN schemas, the risk planes they influence, and their minimum acceptable KER band (kerbandmin) . This automated population of the function catalog is a powerful synergy: the formal contract enables the creation of a rich, trustworthy discovery layer that was previously impossible to build safely. It provides coding agents and AI systems with a high-K, high-E, low-R planning surface, allowing them to generate code and plans that are grounded in the verified ecosafety grammar of the entire constellation . This completes the cycle, turning abstract mathematical guarantees into tangible, practical tools for scaling development and automation safely.

Enforcing Non-Compensating Trade-Offs: The `PlaneWeightsShard` Specification

While the `NonActuatingWorkload` trait defines what constitutes a safe computational kernel, the `PlaneWeightsShard2026v1.aln` specification establishes the rules governing permissible ecological trade-offs. This artifact is the mathematical object that encodes per-plane weights, non-compensation flags, and uncertainty caps, transforming high-level governance intent into a machine-readable format that can be integrated into the Lyapunov residual and CI gates . Its formalization is critical for preventing "eco-degrading trade-offs," where an improvement in one environmental metric might mask a deterioration in another, effectively allowing for "greenwashing" trajectories that fail to meet the constellation's restorative goals . By freezing this shard specification, the EcoNet constellation gains a centralized and authoritative source for its ecological accounting principles.

The `PlaneWeightsShard2026v1.aln` will be defined and stored in the `aln-platform-ecosystem` repository, making it a canonical piece of the ecosafety

grammar . The ALN schema for this shard will contain fields that precisely define the contribution and constraints of each risk plane. Key fields will include:

- **Plane ID:** A unique identifier for the risk plane, such as `CARBON`, `BIODIVERSITY`, `HYDRAULICS`, or `MATERIALS` .
- **Weight (`w_plane`):** A non-negative real number representing the relative importance of this plane in the calculation of the total Lyapunov residual, `V_t = Σ w_j * r_j^2` . Higher weights indicate that violations in this plane have a greater impact on overall system stability.
- **Non-Offsettable Flag:** A boolean flag (e.g., `carbonnonoffsettable = true`) that signals to the system that improvements in other planes cannot be used to compensate for a worsening trend in this specific plane . This is the core mechanism for enforcing non-compensation.
- **Uncertainty Caps:** Production-level bounds on the maximum allowable uncertainty for risk coordinates in this plane (e.g., `r_carbon_unc_max_prod`). These caps help ensure that risk assessments remain conservative and reliable in deployed environments .

The true power of the `PlaneWeightsShard` is unlocked when it is used to define formal, executable non-compensation invariants. These invariants are predicates that are checked during analysis or deployment, forbidding certain classes of undesirable trajectories. The most important invariant is triggered when a non-offsettable plane experiences a negative shift in its risk profile. Formally, a violation occurs if, for a given non-offsettable plane `p`: 1. The risk coordinate `r_p` at timestep `t+1` is greater than its value at timestep `t` (`r_p(t+1) > r_p(t)`). 2. The risk `r_p` has crossed a soft threshold, such as moving from the gold band to the hard band. The consequence of this violation is a direct constraint on the overall system metrics: the global Risk `R` must not decrease (`R(t+1) ≥ R(t)`) and/or the global Eco-Impact `E` must not increase (`E(t+1) ≤ E(t)`) . Since `E = 1 - R`, constraining `E` is often redundant but can be used for additional clarity. This invariant provides a clear, logical rule against optimizing away harm in one area by shifting it elsewhere. For example, an optimizer could not be allowed to increase carbon emissions while simultaneously improving energy efficiency, as this would constitute a worsening of the carbon plane that is then "compensated for" by an improvement in the energy plane, leaving `R` unchanged or even decreasing it artificially. This rule forces the system to confront and mitigate the actual harm, ensuring that the overall trajectory remains genuinely restorative.

The integration of this shard into the ecosystem requires several concrete steps. First, the ALN schema must be finalized and mirrored in the SQLite discovery spine, likely in a dedicated `planeweights` table that is populated alongside `corridordefinition` .

This ensures that both SQL-based queries and ALN-based validators have access to the same authoritative data. Second, the Lyapunov residual update logic must be modified to consult the `PlaneWeightsShard` when calculating `V_t`. The weight `w_plane` for each plane must be read from the shard and used in the summation formula. Third, and most critically, the CI gates and deployment validation logic must be extended to check for non-compensation invariant violations. Before a new `shardinstance` can be promoted to the `PROD` lane, a validator must compare its `RiskVector` with the previous state. If a non-offsettable plane has worsened, the validator must verify that `R` has not decreased and `E` has not increased. If the invariant is violated, the deployment must be rejected or flagged.

A key challenge in this process is the assignment of weights (`w_j`). While the materials do not prescribe a single algorithm for this, the process is central to defining the constellation's priorities. Weights could be determined through a combination of expert consensus, regional ecological assessments, and governance decisions. For instance, a pilot program in a biodiversity hotspot might assign a higher weight to the `BIODIVERSITY` plane than a program focused on industrial hygiene. The `PlaneWeightsShard` could also be versioned, allowing different regions or projects to adopt different weighting schemes while still operating within the broader EcoNet framework. Another potential extension is to use the `PlaneWeightsShard` to store the weight `w_topology` and its associated flag, unifying different types of governance risks under a single conceptual model. This would allow for a holistic view of risk where infrastructural governance issues are treated with the same quantitative rigor as physical environmental factors.

The successful implementation of the `PlaneWeightsShard` has profound implications for the integrity of the entire system. It transforms the Lyapunov residual from a simple sum of risks into a nuanced representation of a complex ecological and governance landscape. By enforcing non-compensation, it closes a critical loophole that could otherwise undermine the system's goals. This grammar-tightening move, much like adding `rtopology` to the residual, shrinks the admissible region in risk space without introducing new control surfaces, thereby increasing the effective K and E scores while keeping R properly bounded . It is a crucial component of the multi-layered governance stack, providing the rules of ecological accounting that guide the behavior of the entire constellation.

Quantifying Governance Integrity: The `rtopology` Coordinate and Lyapunov Integration

A novel and powerful extension to the EcoNet safety model is the introduction of `rtopology`, a governance metric that treats infrastructural integrity as a first-class ecological risk. This concept acknowledges that a well-governed constellation is a prerequisite for achieving genuine ecological safety. By quantifying governance defects and integrating them directly into the Lyapunov residual, `rtopology` creates a powerful feedback loop: poor governance directly impacts system stability, incentivizing good practices across all repositories . This elevates governance from a secondary concern to a primary variable in the system's stability equation, ensuring that the entire infrastructure is robust and reliable.

The formalization of `rtopology` begins with the definition of a raw topology risk score, denoted as I_{topology}. This score is a weighted sum of two primary categories of governance defects observed across the constellation during a given evaluation window, such as a CI run :

$$I_{\text{topology}} = w_{\text{mislabel}} N_{\text{mislabel}} + w_{\text{missing}} N_{\text{missing}}$$

In this formula, N_{mislabel} counts the number of repositories whose declared characteristics conflict with their actual contents—for example, a repo labeled `MATERIAL` containing actuator code, or a repo with `nonactuatingonly=1` depending on an actuator crate . N_{missing} counts the number of repositories that exist but lack a required manifest file, such as `.econet/econetrepoindex.sql` . The weights are assigned such that mislabels are considered more severe than missing manifests (w_{\text{mislabel}} > w_{\text{missing}} \ge 0), reflecting the greater danger posed by a misidentified component compared to an unknown one . This raw score provides a quantitative measure of the constellation's structural health.

The raw score I_{topology} is not used directly; instead, it must be normalized into the `[0,1]` interval to be compatible with the existing ecosafety grammar. This normalization is achieved using the same piecewise linear corridor kernel that is applied to physical risk coordinates like `rcarbon` or `rtox` . This kernel maps the raw score onto a standardized risk coordinate, `rtopology`, where 0 represents perfect governance and 1 represents a catastrophic failure of governance across the constellation. This ensures that `rtopology` behaves consistently with all other risk coordinates, residing in the same mathematical space and adhering to the same semantic rules.

The final and most elegant part of the formalization is the integration of `rtopology` into the Lyapunov residual. The total residual is extended by adding a weighted quadratic term for the governance risk:

$$V_t^{\text{new}} = V_t + w_{\text{topology}} r_{\text{topology}}^2$$

Here, V_t is the original residual composed of physical risk coordinates, and $w_{\text{topology}} \geq 0$ is a non-negative weight that determines the severity of governance failures on system stability . This formulation is mathematically sound because the new term, $w_{\text{topology}} r_{\text{topology}}^2$, is guaranteed to be non-negative and convex, preserving all the desirable properties of the original Lyapunov function. As `rtopology` increases due to mislabels or missing manifests, the total residual V_t^{new} increases, making the system appear less stable and shrinking the admissible action space. This means that until governance issues are resolved, it becomes harder for the system to find valid, deployable states, effectively penalizing poor governance with reduced operational flexibility . This turns governance from an abstract ideal into a concrete, quantifiable constraint.

The practical implementation of this theory relies heavily on Continuous Integration (CI) jobs acting as active auditors of the constellation's topology. These CI jobs would be responsible for computing the raw I_{topology} score by performing audits of the repository structure. A typical workflow for such a job would be: 1. **Scanning:** The CI job iterates through all repos in the constellation, checking for the presence and validity of the `.econet/econetrepoindex.sql` manifest file . 2. **Validation:** It validates the content of each manifest, cross-referencing the declared `roleband` with the repo's actual contents (e.g., using static analysis to detect actuator-related code). It also verifies that `nonactuatingonly` repos only depend on approved, non-actuating crates . 3. **Counting:** It tallies the counts N_{mislabel} and N_{missing} based on its findings. 4. **Computation and Reporting:** Using the predefined weights, it calculates the raw I_{topology} score. It then uses the corridor kernel to derive `rtopology`. Finally, it emits evidence by creating an ALN shard and populating a row in the `shardinstance` and `knowledgeecoscore` tables, recording the `rtopology` value and its impact on the KER metrics for that evaluation window .

This process seamlessly integrates governance into the existing discovery and scoring pipeline. Because the CI auditor itself is a `NonActuatingWorkload`—it reads manifests, computes a risk score, and writes evidence shards without touching actuators—it fits perfectly within the established safety framework . The data it produces is then available to any downstream tool, from human operators using a Kotlin/Android visualizer to see blast-radius warnings to automated systems enforcing deployment gates.

This closes the loop: CI is no longer just a passive checker of code correctness but an active participant in maintaining the constellation's ecosafety and governance integrity, providing a clear incentive for all developers to maintain clean, accurate, and compliant repository manifests.

Building the Governance Foundation: Manifests, Linters, and the Discovery Spine

The formal contracts of `NonActuatingWorkload`, `PlaneWeightsShard`, and `rtopology` are powerful, but they require a robust infrastructure to be effective at scale. This infrastructure consists of three interconnected pillars: standardized repo manifests, automated linting tools to enforce those manifests, and a unified discovery spine to index and query all assets. Together, these components transform a collection of loosely connected repositories into a coherent, governable, and discoverable constellation. The strategic sequence is to first freeze the manifest schema and its ALN mirror, then build the tools to validate it, and finally populate the discovery spine with the resulting evidence, all before developing advanced automation layers .

The cornerstone of this foundation is the `econetrepoindex.sql` shard, a small SQL file placed in the root of each repository (e.g., in a `.econet/` directory) that acts as a self-describing safety contract . This manifest contains essential metadata about the repository, including its name, GitHub slug, `roleband` (e.g., ENGINE, MATERIAL, GOV), primary language, and a description of its purpose . Crucially, it also specifies the `ecosafetybinding`—the version of the ecosafety grammar it conforms to—and its `shardprotocol` . To add finer-grained detail, a companion `econetlayer` table can list internal programming layers (e.g., KERNEL, EDGESCRIPT, MATERIAL) and their associated contracts, such as "SafeKernel" or "NonActuatingWorkload" . This manifest is not just documentation; it is a machine-readable declaration of a repository's role and responsibilities within the larger system.

To elevate these manifests from simple text files to enforceable contracts, a companion tool called `econetrepomanifestlint` must be developed and integrated into the CI pipeline for every repository in the constellation . This binary tool will be hosted in the `EcoNet-index` crate within the SPINE repository. Its sole purpose is to parse a repository's `econetrepoindex.sql` file and validate it against a canonical set of governance rules defined in the central SPINE. The linter will perform checks such as verifying that the `roleband` is one of the predefined values, that the `lanedefault` is a

valid lane (RESEARCH, EXPPROD, PROD), and that the `kertargetK/E/R` values are within expected ranges . More importantly, it will enforce the one-role-band-per-repo invariant and check that a repository marked `nonactuatingonly=1` does not have dependencies on any crates that expose actuator APIs . When a PR modifies a manifest or adds a new repo, the CI job will run `econetrepomanifestlint`. Any violation will cause the CI check to fail, forcing the developer to correct the manifest before merging. This automated enforcement is what makes the manifests a true governance tool, turning them into a live database of truth that is constantly audited and corrected.

With a mechanism for generating and validating manifests in place, the next step is to populate the central discovery spine. The discovery spine is a SQLite database that serves as the backbone for all cross-repo queries and discovery operations . Its schema, comprising tables like `repo`, `repofile`, `alnschema`, `alnparticle`, `shardinstance`, and `knowledgeecoscore`, is already largely defined . The task is to finalize this schema, create ALN schema mirrors for uniform access, and write scripts to crawl the constellation repositories, read their manifests, and seed the spine with structured data . For example, a script would iterate through all repos, read their `econetrepoindex.sql`, and insert a row into the `repo` table. It would then walk the `qpudatashards` directory, read the ALN files, and populate the `repofile`, `alnschema`, `alnparticle`, and `shardinstance` tables with metadata about each shard, linking it back to its source repository and particle definition . This process creates a permanent, indexed record of the entire constellation's assets, their properties, and their governance status.

On top of this evidence base, a final layer of intelligence is added by creating an `agentfunctioncatalog` table. This table acts as a searchable index of all safe, discoverable functions in the system. It is populated automatically by scanning all crates for implementations of the `NonActuatingWorkload` trait . Each entry in the catalog would map a specific function to its properties: its name, the crate it belongs to, the repository it comes from, its `Input` and `Output` types (mapped to ALN schemas), the risk planes it operates on, and its minimum acceptable `KER` band (`kerbandmin`) . This catalog is the key to enabling AI-chat compatible discovery. An agent can now formulate a query like, "Find all `MATERIAL` repos with water-compatible substrates," and the query engine can translate this into a series of SQL joins against the discovery spine and the `agentfunctioncatalog`. It can filter for repos with `roleband='MATERIAL'`, find their `econetrepoindex` entries, and then look up all `NonActuatingWorkload` implementations that produce relevant output types. The results would be a filtered list of high-K, high-E, low-R candidates, grounded in the verified manifest data and trait implementations . This entire stack—from manifests and linters to the discovery spine

and function catalog—provides the necessary groundwork to build powerful, safe, and effective automation tools without compromising the system's integrity.

Synthesis and Strategic Pathway for Constellation-Wide Implementation

The body of work detailed throughout this report outlines a comprehensive and coherent strategy for formalizing the EcoNet SPINE repository to create a high-integrity foundation for governance and automation. The core insight is the establishment of a multi-layered, self-reinforcing governance stack that progresses from immutable grammar to automated enforcement and finally to actionable intelligence. This phased approach is designed to maximize safety (minimizing R) while simultaneously increasing knowledge (K) and eco-impact (E), aiming to hold the constellation within a stable operating regime of $K \approx 0.95$, $E \approx 0.92$, and $R \approx 0.12$. The strategic pathway prioritizes the creation of foundational contracts before scaling discovery and automation tools, ensuring that every subsequent layer is built upon a verifiable and consistent basis .

The first and most critical layer is the **Foundation Layer**, which comprises the immutable grammatical contracts of the system. This includes the `NonActuatingWorkload` trait, which defines the syntax and semantics of safe computation, and the `PlaneWeightsShard2026v1.aln`, which codifies the rules of ecological accounting and non-compensation . These artifacts are not merely documents or guidelines; they are machine-verifiable contracts implemented in Rust and ALN. The `NonActuatingWorkload` trait, with its stringent requirements for Boundedness, Monotonicity, and the Safestep invariant, provides a mathematical guarantee that any implementation will behave predictably and safely . The `PlaneWeightsShard` provides the authority for determining which ecological trade-offs are permissible, preventing deceptive optimization paths that could undermine the system's restorative goals . The introduction of `rtopology` extends this principle to governance, treating infrastructural integrity as a first-class risk variable that directly impacts system stability .

The second layer is the **Execution Layer**, where these abstract contracts are put into practice through automation. The primary mechanism here is the CI pipeline. CI jobs are transformed from passive checkers of code correctness into active validators of the entire ecosafety and governance model. A `econetrepomanifestlint` tool, run in CI for every repository, enforces the `econetrepoindex` manifests, turning them from documentation into live, auditable contracts . Simultaneously, topology auditor jobs

compute the `I_topology` score, normalize it to `rtopology`, and feed this governance metric back into the system's evidence base . This layer is responsible for generating the evidence—the populated discovery spine—that reflects the current state of the constellation, including the properties of its assets and the health of its governance .

The third layer is the **Evidence Layer**, embodied by the unified discovery spine. This is a central SQLite and ALN database that serves as the permanent, indexed record of the entire constellation . It aggregates metadata from every repository, mapping ALN files to particles, tracking the properties of every `shardinstance`, and recording governance scores in `knowledgeecoscore` . This spine is the single source of truth for the entire system, providing a consistent and queryable surface for all discovery and analysis tasks. The fidelity of this evidence layer is paramount; its quality depends entirely on the rigor of the manifests and the thoroughness of the CI-driven population scripts.

Finally, the fourth layer is the **Application Layer**, which includes all the tools and agents that consume the evidence from the spine. This includes the `econetrepomanifest` crate, FFI interfaces for Lua and Kotlin, and AI-chat optimized query functions . With the foundation, execution, and evidence layers in place, this application layer can be built with a high degree of confidence. Agents can now ask sophisticated questions—"list all `MATERIAL` repos working on water-compatible substrates"—and receive answers that are grounded in a verified, high-integrity dataset . The `agentfunctioncatalog`, automatically generated from `NonActuatingWorkload` implementations, provides a searchable index of all safe, discoverable kernels, enabling powerful and safe code generation and planning .

The immediate and highest-yield upgrade pathway is a disciplined, sequential implementation of these layers. The recommended path is: 1. **Freeze the Contracts:** Finalize the `NonActuatingWorkload` trait and its associated invariants, and define the `PlaneWeightsShard2026v1.aln` schema. Implement these in existing non-actuating crates and write the requisite unit tests. 2. **Harden Governance Tools:** Develop and deploy the `econetrepomanifestlint` tool and integrate it into the CI of all constellation repositories. Begin requiring manifests and linting as a gate for all contributions. 3. **Populate the Spine:** Create the migration scripts and crawlers to populate the SQLite and ALN discovery spine with data from the newly governed repositories. 4. **Build the Function Catalog:** Generate the `agentfunctioncatalog` table by scanning for `NonActuatingWorkload` implementations and map them to their ALN schemas and risk planes.

By following this strategic pathway, the EcoNet constellation can achieve its ambitious goals of scaling discovery and automation safely. This approach ensures that every new

tool and capability is built upon a solid, verifiable foundation, progressively tightening the grammar of the system without introducing new sources of risk. It provides a clear roadmap for maturing from a collection of independent research projects into a unified, ecologically restorative, and highly intelligent computational infrastructure.

Reference

1. EcoNet: A new software for ecological modeling, simulation and ... https://www.researchgate.net/publication/248413934_EcoNet_A_new_software_for_ecological_modeling_simulation_and_network_analysis
2. Security and Hardening Guide | SLES 15 SP6 - SUSE Documentation https://documentation.suse.com/sles/15-SP6/single-html/SLES-security/index.html
3. Society For Mathematical Biology 2012 | PDF - Scribd https://www.scribd.com/document/129250411/Society-for-Mathematical-Biology-2012
4. Advanced Intelligent Computing Technology and Applications https://link.springer.com/content/pdf/10.1007/978-981-97-5591-2.pdf
5. Pinet: Optimizing hard-constrained neural networks with orthogonal ... https://arxiv.org/abs/2508.10480
6. PhiNets: a scalable backbone for low-power AI at the edge - arXiv https://arxiv.org/abs/2110.00337
7. Security and Hardening Guide | SLES 15 SP7 - SUSE Documentation https://documentation.suse.com/sles/15-SP7/single-html/SLES-security/
8. A Survey of Vibe Coding with Large Language Models - arXiv https://arxiv.org/html/2510.12399v1
9. [PDF] EMNLP 2021 - ACL Anthology https://aclanthology.org/2021.emnlp.handbook.pdf
10. [PDF] Advanced Biomedical Engineering - ResearchGate https://www.researchgate.net/profile/Alistair-Mcewan-2/publication/260294332_Advanced_Biomedical_Engineering/links/00b495387033ea70d5000000/Advanced-Biomedical-Engineering.pdf
11. 软件工程/系统与控制/多代理系统/数学软件2026_4_14 http://arxivdaily.com/thread/78766

12. Arxiv今日论文| 2026-03-20 - 闲记算法 http://lonepatient.top/2026/03/20/arxiv_papers_2026-03-20
13. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=150&show=1000
14. au:Zhang_D in:cs - SciRate Search https://scirate.com/search?q=au:Zhang_D+in:cs
15. Artificial Intelligence in HCI - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-93429-2.pdf
16. Intelligent Systems Applications in Software Engineering https://link.springer.com/content/pdf/10.1007/978-3-030-30329-7.pdf
17. glove.6B.100d.txt-vocab.txt - CodaLab Worksheets https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt
18. Dicionario portugues - Academia.edu https://www.academia.edu/32592435/Dicionario_portugues
19. Published Password Lists: 1 - Ineapple https://ineapple.com/known_pass1
20. Simulation and Modeling Methodologies, Technologies and ... https://link.springer.com/content/pdf/10.1007/978-3-030-01470-4.pdf